Architecture Documentation: High-Speed Custom UDP Protocol

1. Protocol Overview

This protocol is a lightweight, high-performance, user-space reliability layer wrapped entirely inside standard UDP datagrams. Designed specifically for closed local Wi-Fi networks, it bypasses the heavy state-machine and minimum size requirements of TCP while utilizing standard UDP to traverse NAT routers and OS kernels without requiring root privileges.

The protocol operates on a **Sliding Window with Piggybacked Retransmissions**, eliminating the need for periodic ACK bitfields and reducing the custom header to an absolute minimum of 7 bytes.

2. The 7-Byte Custom Header Specification

The header is placed immediately following the standard 8-byte UDP header. All data integrity (corruption checking) is deferred to the OS-level UDP Checksum calculation.

Field Name	Size	Purpose & Mechanics
App ID	**2 Bytes** (16 bits)	**Protocol Guard & Multiplexer.** Instantly filters out random Wi-Fi noise. Allows a single UDP socket to manage multiple independent client sessions or sub-streams (e.g., separating file transfer data from control data).
Flags	**1 Byte** (8 bits)	**Packet State Machine.** Determines packet intent and batch boundaries (detailed below).
Sequence Number	**4 Bytes** (32 bits)	**Reassembly Anchor.** Tracks absolute ordering. Used by the

Field Name	Size	Purpose & Mechanics
		receiver to detect gaps in a batch and map out-of-order piggybacked packets back into their correct buffer slots.

1-Byte Flag Breakdown (Bitwise Map)

The 8 bits of the Flag byte control the entire network state without requiring separate control packets:

- **Bit 0 (Type):** `0` = Data Packet (Contains application payload) | `1` = NACK Payload (Receiver requesting missing sequence numbers).
- **Bit 1 (State):** `0` = Fresh Data | `1` = Piggybacked Retransmission (Alerts receiver that this sequence number belongs to a past batch).
- **Bit 2 (Batch Control):** `0` = Mid-Batch Packet | `1` = `END_OF_BATCH` (Forces receiver to evaluate missing sequences and fire a NACK).
- **Bits 3 to 7:** Reserved (Set to `0` for future extensions like compression or encryption).

3. Transmission & Recovery Strategies

Strategy A: Variable Batch Sizing (Application-Level Framing)

Instead of continuous unstructured streams, data is chunked into logical batches. The batch size is dynamic, allowing the sender to adapt to network conditions and payload sizes.

- **Mechanism:** The sender blasts a burst of packets. The final packet of that burst has the `END_OF_BATCH` flag (Bit 2) set to `1`.
- **Advantage:** Prevents massive RAM bloat. The application only needs to buffer the current active batch rather than the entire file, while dynamically scaling batch sizes based on real-time Wi-Fi congestion.

Strategy B: Payload NACKs (Negative Acknowledgments)

The continuous 32-bit ACK bitfield has been removed in favor of a deferred, payload-based NACK system to maximize forward Wi-Fi bandwidth.

- **Mechanism:** The receiver passively accepts data. When it receives an `END_OF_BATCH` flag, it scans its sequence buffer. If packets are missing (e.g., expected #50 but only has up to

#49 and #51), it generates a single Retransmission Request (Bit 0 = `1`). The payload of this NACK packet contains an array of the exact missing Sequence Numbers.
- **Timeout Safety:** If the actual `END_OF_BATCH` packet drops, the receiver will never know the batch finished. A strict async socket timeout (e.g., 15–20ms of radio silence) is implemented to force a NACK generation based on the highest received sequence number.

Strategy C: Sliding Window with Piggybacked Retransmissions (The Master Strategy)

To completely eliminate "stop-and-wait" latency, the protocol never pauses the Wi-Fi radio to fix dropped packets.

- **Mechanism:** If Batch 1 drops packets #12 and #45, the receiver fires a NACK. The sender does not stop to send just #12 and #45. Instead, it immediately begins transmitting Batch 2 (Sequence #100–200) and **injects** #12 and #45 directly into the new stream with the Piggybacked flag (Bit 1) set to `1`.
- **Advantage:** Zero idle radio time. The network interface card blasts data continuously at maximum hardware speed.

4. Memory Lifecycle & Buffer Management

Because packets are delivered out-of-order and retransmitted inside future batches, the memory architecture follows strict rules:

- **Sender Lifecycle:** A batch is only cleared from the sender's RAM when a NACK payload from the receiver confirms that zero packets from that specific batch are missing.
- **Receiver Lifecycle:** The receiver utilizes pre-allocated memory buffers. As piggybacked packets arrive, they are instantly slotted into their specific byte-offsets from the previous batch, completing the old batch and handing it to the application layer while the new batch continues to stream in.